

Generic external axis over Modbus TCP — protocol

One register map for any external axis a PLC fronts: a linear track carrying the robot (Pamet: Kinco PLC → Lichuan servo), a rotary positioner carrying the part, or any drive behind a PLC that can serve Modbus TCP. rw2 owns the axis through this map alone. The robot controller's own external-axis coupling is never used (owner ruling 2026-09-28/29). The map is the contract between the driver in this repository and the PLC program; both sides are written against this file.

Lineage: epic-065 external-axis-motion (Delta VFD-C2000 positioner, `delta-positioner` `DeltaRegisters.cs`, FR-11 dead-commander watchdog) and epic-065 feature-007 (Pamet track directly from the Kinco PLC).

Transport

Item	Value
Transport	Modbus TCP, port 502 (configurable)
Unit id	1 (configurable)
Register type	Holding registers only (FC03 read, FC06/FC16 write). No coils: one register file is simpler to map in every PLC.
Word order for 32-bit values	low word first (register n = bits 0–15, register n+1 = bits 16–31), two's complement
Position unit	0.001 mm for a linear axis, 0.001° for a rotary axis (<code>kind</code> decides; the map is the same)
Velocity unit	0.001 unit/s; acceleration 0.001 unit/s ²
Map version	register 114 = 1 for this document

Command block — written by the driver, read by the PLC (holding registers 0–11)

Addr	Name	Type	Meaning
0	Command	bitfield	bit 0 Enable (level: 1 = servo on). bit 1 Home , bit 2 MoveAbsolute , bit 3 MoveVelocity , bit 4 Stop , bit 5 Reset — edge-triggered: the PLC acts on 0→1 and the driver clears the bit after <code>CommandAck</code> echoes <code>CommandSeq</code> . Stop has priority over every other bit in the same word.
1	CommandSeq	uint16	Incremented by the driver with every command write (never 0). The PLC copies it to <code>CommandAck</code> when it has accepted the command.
2–3	TargetPosition	int32	Target for MoveAbsolute, in position units.
4–5	Velocity	int32	Speed for MoveAbsolute (magnitude) and MoveVelocity (signed: sign = direction).
6–7	Acceleration	int32	Ramp for the next move; 0 = PLC default.

Addr	Name	Type	Meaning
8	Heartbeat	uint16	FR-11: the driver writes an incrementing value (1...65535, never 0) at ≥ 5 Hz for the whole connection lifetime.
9	LeaseOwner	uint16	FR-11 advisory lease: 0 = unowned; else the station-unique owner id of the driver holding the axis.
10	WatchdogFault	uint16	0 healthy; 1 = the PLC tripped the watchdog. Cleared only by the driver writing 0. Written by the PLC on trip.
11	WatchdogTrips	uint16	Trip counter since PLC power-up. PLC-owned, read-only for the driver.

Status block — written by the PLC, read by the driver (holding registers 100–114)

Addr	Name	Type	Meaning
100	State	uint16	0 Disabled · 1 Standstill · 2 Homing · 3 DiscreteMotion · 4 ContinuousMotion · 5 (reserved) · 6 Stopping · 7 ErrorStop — the numbers of the SDK <code>AxisState</code> enum.
101	Flags	bitfield	bit 0 Homed · bit 1 InPosition · bit 2 LimitMin · bit 3 LimitMax · bit 4 HomeSensor · bit 5 DriveReady · bit 6 Moving
102–103	ActualPosition	int32	Position units. Valid only while <code>Homed</code> is set (an absolute encoder keeps it across power cycles; the map does not care which).
104–105	ActualVelocity	int32	Signed, 0.001 unit/s.
106	FaultCode	uint16	0 none · 1 drive fault · 2 limit tripped · 3 following error · 4 watchdog · 5 homing failed · 6 communication to drive lost · 100+ vendor-specific (documented per PLC).
107	CommandAck	uint16	Echo of the last accepted <code>CommandSeq</code> .
108–109	TravelMin	int32	Machine limit, published by the PLC. The driver refuses targets outside <code>TravelMin..TravelMax</code> ; it never clamps.
110–111	TravelMax	int32	
112–113	MaxVelocity	int32	Machine limit, published by the PLC. The driver refuses faster speeds.
114	MapVersion	uint16	1. The driver refuses to attach to any other value.

Limits come from the machine through this block. If a PLC cannot publish them (all three zero), the driver uses the values from its own configuration and logs that it did; it never invents a number.

Command semantics

- **Enable** (level): `Command.bit0 = 1` → PLC energises the drive; State leaves Disabled. `bit0 = 0` → drive off, State = Disabled. A move with Enable low is ignored and answered by `FaultCode = 0`, `State = Disabled`; the driver rejects it before writing (SDK `MotionError.Busy / NotHomed` as applicable).

- **Home** (edge): PLC runs its own homing sequence; State = Homing until done, then Standstill with Homed . The driver treats Home as callable from Disabled (it is the energise act: the driver sets Enable first). Failure → ErrorStop, FaultCode = 5 .
- **MoveAbsolute** (edge): requires Homed ; PLC moves to TargetPosition at Velocity ; State = DiscreteMotion, then Standstill with InPosition . Target outside travel → the driver refuses (SDK OutOfRange) and writes nothing.
- **MoveVelocity** (edge): continuous motion at signed Velocity until Stop or a limit; State = ContinuousMotion.
- **Stop** (edge, priority): decelerate and hold; State = Stopping → Standstill. The driver sends Stop on the priority lane (see FR-11) and expects motion to cease within 200 ms of the write.
- **Reset** (edge): clears FaultCode and leaves ErrorStop → Disabled (Enable must be re-asserted). Does not clear WatchdogFault (register 10 is cleared explicitly) and does not touch Homed .
- **Acknowledge**: a command is accepted when CommandAck == CommandSeq . The driver waits ≤ 500 ms for the ack; no ack → SDK CommunicationLost for the command (the heartbeat keeps running).

FR-11 — dead-commander watchdog and advisory lease (verbatim from epic-065)

Motion must cease when the commander **dies**, not only when it calls Stop. Modbus TCP has no liveness semantics; a dead commander is indistinguishable from an idle one. Therefore:

- The driver runs a **connection-lifetime heartbeat** per axis, started at connect, stopped deliberately at disconnect, independent of motion, writing Heartbeat at ≥ 5 Hz. Heartbeat traffic shares the channel with move traffic; heartbeat deferral is **bounded at 200 ms** so a long move cannot starve the beat.
- The PLC **arms** the watchdog on the **first change** of Heartbeat it observes after power-up (so an idle PLC is not faulted one second after boot), then **trips** when Heartbeat has not changed for 1 s: Enable dropped, velocity zeroed, State = ErrorStop, FaultCode = 4, WatchdogFault = 1, WatchdogTrips += 1. A trip does **not** touch Homed (recovery = Reset + re-command, no re-home) and re-asserts the limit functions. After a trip the network disarms until WatchdogFault has been written 0 and a new beat arrives.
- **Advisory lease**: before attaching, the driver reads LeaseOwner . 0 → take it (write own id). Own id → reattach. Foreign id → sample Heartbeat across one expiry window (1 s); unchanged → the lease has expired, take it; changed → refuse (SDK LeaseHeld) and retry at 1 Hz until the configured lease timeout. "Advisory" is stated plainly: Modbus has no compare-and-swap, so two drivers starting in the same window can both pass; the watchdog bounds the consequence. On a clean disconnect the driver writes LeaseOwner = 0 if it still holds it.
- Owner id is a station-unique non-zero 16-bit value from host configuration.
- The heartbeat's paired read of registers 10–11 doubles as the status cadence for an idle axis.
- Stop and cancellation take a **priority lane** on the channel that preempts queued move traffic (≤ 200 ms to a halted axis).

What the PLC program must do (checklist for the ladder / structured-text author)

1. Serve holding registers 0–11 and 100–114 on unit id 1, port 502; refuse nothing, ignore writes to PLC-owned registers (11, 100–114).
2. Publish MapVersion = 1, TravelMin/Max, MaxVelocity from the axis parameters at start-up.

3. Mirror the drive: `State`, `Flags`, `ActualPosition`, `ActualVelocity`, `FaultCode` every PLC cycle.
4. Execute the command semantics above; echo `CommandSeq` into `CommandAck` on acceptance.
5. Implement the watchdog exactly as FR-11 states (arm on first change, 1 s stall, trip actions, disarm until cleared).
6. Keep the project in a git repository next to this file's copy (epic-065 risk R-1: an unversioned PLC program is a drive that answers every register and moves nothing after replacement).

Reference: the Delta positioner ladder this generalises

```
M0 -> M1025 (RUN) + M1040 (servo on)
M4 -> edges: MOV 0/1 D1060 (speed/position mode); FREQ D110 D111 D112
M5 -> M1026 (direction: OFF = forward, ON = reverse)
M6 + edge X7 -> DSUB D1051 D120 D122; DMOV D1051 D120 (home latch)
D130 heartbeat · D131 lease owner · D132 latched watchdog fault · D133 trip counter
```

(`delta-positioner/src/RocketWelder.SDK.Devices.Motion.Delta/DeltaRegisters.cs`, epic-065 `current-state.md`.)